

Aula 009

1) Java

a)

```
J Main.java x + Console
1 class Animal {
2     String som() { return "..."; }
3 }
4
5 class Cachorro extends Animal {
6     String som() { return "au"; }
7 }
8
9 public class Main {
10     public static void main(String[] args) {
11         Animal x = new Cachorro();
12         System.out.println(x.som());
13     }
14 }
```

au

b) Decisão na execução: som() é sobrescrito e a JVM usa o tipo real do objeto (Cachorro).

2) C++

a)

```
G Main.cpp x + Console
1 #include <iostream>
2 using namespace std;
3
4 struct A {
5     void f() { cout << "A"; }
6 };
7
8 struct B : A {
9     void f() { cout << "B"; }
10 };
11
12 int main() {
13     B b; A *p = &b;
14     p->f();
15     cout << endl;
16     return 0;
17 }
```

A

b) Decisão na compilação: f() não é virtual, então vale o tipo do ponteiro (A*).

3) Java

a)



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines two classes: 'A' and 'B'. Class 'A' has a 'String' attribute 'nome' with the value 'A' and a 'getNome()' method that returns 'nome'. Class 'B' extends 'A' and also has a 'String' attribute 'nome' with the value 'B' and a 'getNome()' method that returns 'nome'. The 'Main' class has a 'main' method that creates an instance of 'B' (x) and prints 'x.nome' followed by 'x.getNome()'. The console output shows 'A B'.

```
1 class A { String nome = "A";  
2     String getNome() { return nome; } }  
3  
4 class B extends A { String nome = "B";  
5     String getNome() { return nome; } }  
6  
7 public class Main {  
8     public static void main(String[] args) {  
9         A x = new B();  
10        System.out.println(x.nome + " " + x.getNome());  
11    }  
12 }
```

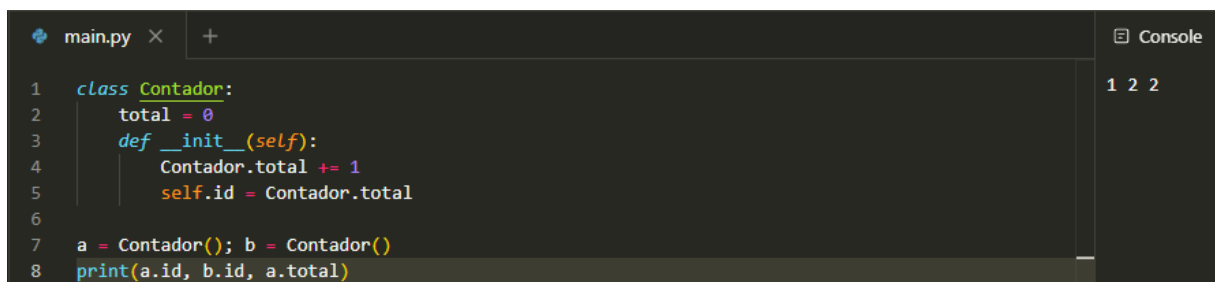
Console: A B

A B

b) Decisão dividida: x.nome usa o tipo declarado (A) na compilação; getNome() usa o tipo real (B) na execução.

4) Python

a)



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a class 'Contador' with a class attribute 'total' set to 0. The '__init__' method increments 'total' by 1 and sets 'self.id' to 'Contador.total'. Two instances 'a' and 'b' are created, and their 'id' and 'total' attributes are printed. The console output shows '1 2 2'.

```
1 class Contador:  
2     total = 0  
3     def __init__(self):  
4         Contador.total += 1  
5         self.id = Contador.total  
6  
7 a = Contador(); b = Contador()  
8 print(a.id, b.id, a.total)
```

Console: 1 2 2

1 2 2

b) Decisão na execução: total é atributo da classe, compartilhado entre as instâncias.

5) Java

a)



```
1  class A {
2      static String quem() { return "A"; }
3  }
4
5  class B extends A {
6      static String quem() { return "B"; }
7  }
8
9  public class Main {
10     public static void main(String[] args) {
11         A x = new B();
12         System.out.println(x.quem());
13     }
14 }
```

Console: A

A

b) Decisão na compilação: método static não é sobrescrito, só ocultado; vale o tipo declarado (A).

6) Go

a)



```
1  package main
2
3  import "fmt"
4
5  type Animal struct{}
6
7  func (Animal) Som() string { return "..."}
8
9  func (a Animal) Falar() string {
10     return "faz " + a.Som()
11 }
12
13 type Cao struct{ Animal }
14
15 func (Cao) Som() string { return "au"}
16
17 func main() {
18     fmt.Println(Cao{}.Falar(), Cao{}.Som())
19 }
```

Console: faz ... au

faz ... au

b) Decisão na compilação: Falar() recebe um Animal, então a.Som() é sempre o de Animal; embedding em Go não é herança com métodos virtuais.